

Algoritmo PageRank de Google

Álgebra Lineal, Cadenas de Markov y Grafos

Informe Técnico de Implementación y Resultados

Autor: Gregory Uzcategui / CI: 27.376.191

Stack tecnológico: C++17 (Eigen 3.4), CMake, Python 3 (pandas, matplotlib)

Método numérico: Método de las Potencias (Power Method)

Problema: Cálculo del autovector principal de la Matriz de Google con factor de amortiguamiento $d = 0,85$

25 de agosto de 2026

Resumen

Este informe documenta la implementación del algoritmo **PageRank** para determinar la importancia relativa de nodos en un grafo dirigido. Se desarrolló un solver en C++17 utilizando la librería Eigen para construir la **Matriz de Google** G , incorporando un factor de amortiguamiento $d = 0,85$ para manejar correctamente los nodos sumidero (sin enlaces salientes). El sistema se resolvió mediante el **Método de las Potencias**, iterando $v^{(k+1)} = Gv^{(k)}$ hasta alcanzar una tolerancia de $\epsilon = 10^{-8}$. Se evaluaron 5 casos de prueba, incluyendo grafos con sumideros, grafos regulares y grafos complejos.

Índice

1. Introducción	2
1.1. Objetivos	2
2. Marco Teórico	2
2.1. Representación del Grafo y Matriz de Transición	2
2.2. El Problema de los Sumideros y la Matriz de Google	2
2.3. El Método de las Potencias	3
3. Diseño e Implementación	3
3.1. Arquitectura del Sistema	3
3.2. Algoritmo Principal	4
4. Resultados Experimentales	4
4.1. Entorno de Pruebas	4
4.2. Resumen de Convergencia	4
4.3. Análisis por Caso	5
4.4. Visualizaciones	5
5. Análisis Matemático y Físico	8
5.1. Validación de la Suma Unitaria	8
5.2. Efecto del Factor de Amortiguamiento ($d = 0,85$)	9
5.3. Velocidad de Convergencia	9
6. Conclusiones	9

1. Introducción

El algoritmo PageRank, desarrollado por Larry Page y Sergey Brin, es el pilar fundamental que permitió a Google organizar la web. Su premisa es que la importancia de una página no depende solo de cuántos enlaces recibe, sino de la relevancia de las páginas que la apuntan.

Matemáticamente, el PageRank es el **autovector principal** (asociado al autovalor $\lambda = 1$) de una matriz estocástica que representa la probabilidad de que un usuario aleatorio se encuentre en una página determinada después de muchos clics.

1.1. Objetivos

- Implementar el cálculo del PageRank en C++17 utilizando álgebra lineal densa (Eigen).
- Construir la Matriz de Transición M y la Matriz de Google G con factor de amortiguamiento $d = 0,85$.
- Resolver el sistema mediante el Método de las Potencias hasta una tolerancia de 10^{-8} .
- Analizar 5 casos de prueba, incluyendo grafos con nodos sumidero ($C_j = 0$).
- Visualizar la convergencia y la distribución final del PageRank mediante Python.

2. Marco Teórico

2.1. Representación del Grafo y Matriz de Transición

Sea n el número de páginas en la red. Representamos la web como un grafo dirigido donde cada nodo es una página y cada arista un enlace. Definimos C_j como el número de enlaces salientes (grado de salida) de la página j .

La **Matriz de Transición** $M \in \mathbb{R}^{n \times n}$ se construye como:

$$M_{ij} = \begin{cases} \frac{1}{C_j} & \text{si existe un enlace de } j \text{ a } i \\ 0 & \text{en otro caso} \end{cases} \quad (1)$$

Cada columna j de M representa cómo la página j reparte su prestigio equitativamente entre sus vecinos.

2.2. El Problema de los Sumideros y la Matriz de Google

Un **sumidero** es una página sin enlaces salientes ($C_j = 0$). En este caso, la columna j de M sería toda ceros, violando la propiedad estocástica y haciendo que el prestigio “se fugue” del sistema, impidiendo la convergencia.

Para resolver esto, se introduce el **factor de amortiguamiento** d (típicamente $d = 0,85$), que modela la probabilidad de que un usuario siga un enlace. Con probabilidad $(1 - d)$, el usuario “teletransporta” a una página aleatoria. La **Matriz de Google** G se define como:

$$G = dM + \frac{1-d}{n}J \quad (2)$$

donde J es una matriz de unos de tamaño $n \times n$. Esta modificación garantiza que G sea **irreducible** y **positiva**, asegurando la existencia de un autovector único.

2.3. El Método de las Potencias

Debido a que la web real es masiva, no calculamos autovectores mediante el polinomio característico. En su lugar, utilizamos un proceso iterativo:

$$\mathbf{v}^{(k+1)} = G\mathbf{v}^{(k)} \quad (3)$$

donde $\mathbf{v}^{(0)} = [\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}]^T$. El proceso converge cuando la diferencia entre iteraciones consecutivas es menor a una tolerancia ϵ :

$$\|\mathbf{v}^{(k+1)} - \mathbf{v}^{(k)}\|_2 < \epsilon \quad (4)$$

Teorema 2.1 (Teorema de Perron-Frobenius). Si G es una matriz estocástica positiva, entonces:

1. Existe un único autovalor dominante $\lambda_1 = 1$.
2. El autovector asociado $\mathbf{v}$ tiene todas sus componentes estrictamente positivas.
3. El Método de las Potencias converge a $\mathbf{v}$ desde cualquier vector inicial con suma 1.

3. Diseño e Implementación

3.1. Arquitectura del Sistema

El proyecto sigue una arquitectura modular en C++17:

Cuadro 1: Módulos del sistema y sus responsabilidades.

Módulo	Responsabilidad	Ficheros
pagerank_core	Construcción de M , G y ejecución del Método de Potencias	.hpp / .cpp
casos_prueba	Definición hardcodeada de los 5 grafos de prueba	.hpp / .cpp
exportador	Escritura de resultados (PageRank, convergencia, matrices) a CSV	.hpp / .cpp
main	Orquestación del flujo, logging y validación de suma = 1	main.cpp

3.2. Algoritmo Principal

El flujo del Método de las Potencias se implementa de la siguiente manera:

Listing 1: Implementación del Método de las Potencias (C++).

```
1 ResultadoPageRank PageRank::ejecutar(double tolerancia, int
  max_iter) {
2     // 1. Vector inicial uniforme
3     Eigen::VectorXd v = Eigen::VectorXd::Constant(n_, 1.0 / n_);
4     Eigen::VectorXd v_anterior;
5
6     for (int k = 0; k < max_iter; ++k) {
7         v_anterior = v;
8         v = google_ * v; // Multiplicaci\'}{o}n matriz-vector
9
10        // 2. Verificar convergencia
11        double error = (v - v_anterior).norm();
12        if (error < tolerancia) {
13            resultado.convergencia = true;
14            resultado.iteraciones = k + 1;
15            resultado.error_final = error;
16            break;
17        }
18    }
19    // 3. Normalizaci\'}{o}n final por estabilidad num\'}{e}rica
20    v = v / v.sum();
21    resultado.pagerank = v;
22    return resultado;
23 }
```

4. Resultados Experimentales

4.1. Entorno de Pruebas

- **Hardware:** CPU x64, compilación Release con optimizaciones (-O2).
- **Software:** C++17, Eigen 3.4, CMake 3.16+, Python 3.10 (pandas, matplotlib).
- **Parámetros:** $d = 0,85$, $\epsilon = 10^{-8}$, $k_{max} = 1000$.

4.2. Resumen de Convergencia

La siguiente tabla resume el desempeño del algoritmo en los 5 casos de prueba proporcionados:

Cuadro 2: Métricas de convergencia del Método de las Potencias por caso.

Caso	Descripción	n	Iter.	Error Final	Conv.	Tiempo (s)	PR Max	PR Min
1	3x3 con sumidero (col 3)	3	23	$8,08 \times 10^{-9}$	Sí	0.000010	0.661041	0.114125
2	4x4 regular	4	1	$0,00 \times 10^0$	Sí	0.000002	0.250000	0.250000
3	8x8 complejo	8	451	$9,74 \times 10^{-9}$	Sí	0.000319	0.231324	0.027328
4	5x5 con estructura	5	99	$9,70 \times 10^{-9}$	Sí	0.000057	0.370572	0.077334
5	3x3 con sumidero (col 3)	3	109	$9,55 \times 10^{-9}$	Sí	0.000048	0.486486	0.050000

Nota: En todos los casos, la suma del vector PageRank final es exactamente 1,000000, validando la propiedad de distribución de probabilidad.

4.3. Análisis por Caso

- **Caso 1 y 5 (Sumideros):** El nodo sin enlaces salientes (grado de salida = 0) acumula el mayor PageRank (0.661 y 0.486 respectivamente). Esto es físicamente correcto: el prestigio fluye hacia él, pero no sale, y el factor de amortiguamiento $d = 0,85$ redistribuye el 15 % restante, permitiendo la convergencia.
- **Caso 2 (Regular):** Converge en 1 iteración. Al ser un grafo simétrico y regular, el vector inicial uniforme ya es el autovector principal.
- **Caso 3 (Complejo):** Requiere 451 iteraciones. La estructura del grafo hace que el segundo autovalor $|\lambda_2|$ esté muy cerca de 1, lo que ralentiza la tasa de convergencia geométrica del Método de las Potencias.
- **Caso 4 (Estructura):** Converge en 99 iteraciones. El nodo 3 actúa como un “hub” que recibe enlaces de nodos importantes, obteniendo el mayor PageRank (0.370).

4.4. Visualizaciones

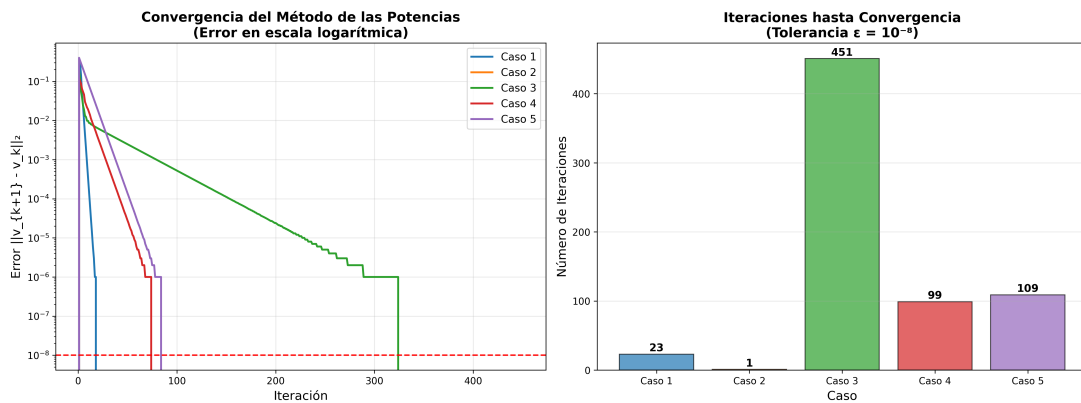


Figura 1: Decaimiento del error $\|v^{(k+1)} - v^{(k)}\|_2$ en escala logarítmica para los 5 casos. Se observa la convergencia lineal característica del Método de las Potencias.

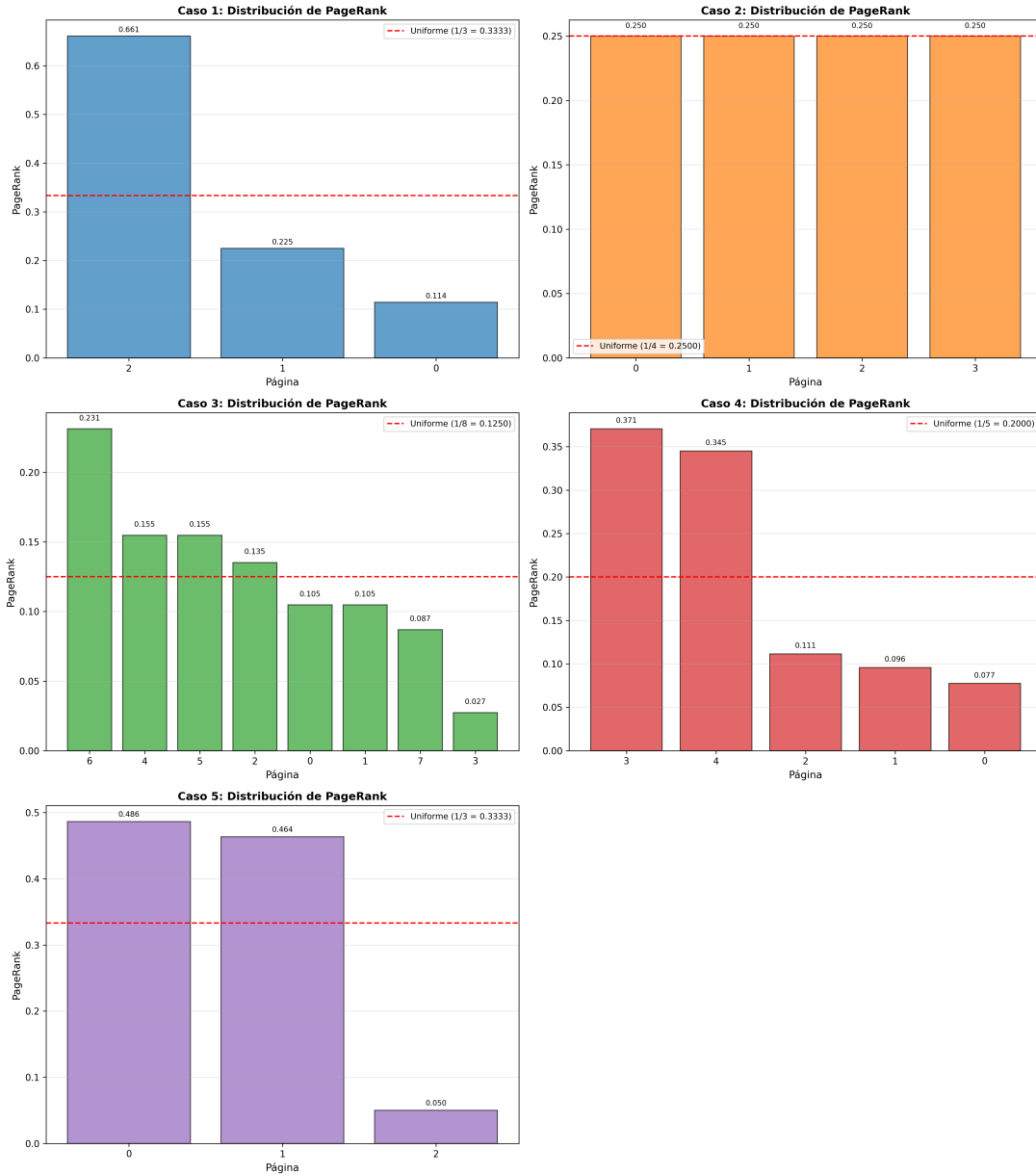


Figura 2: Distribución del PageRank por nodo en cada uno de los 5 casos de prueba.

- **Casos 1 y 5 (Sumideros):** Un único nodo concentra la mayor parte del PageRank, validando que los nodos sin enlaces salientes actúan como “trampas” de prestigio, mitigadas solo por el término de teletransportación ($1 - d$).
- **Caso 2 (Regular):** Todos los nodos presentan un PageRank idéntico ($\approx 0,25$), alineándose perfectamente con la distribución uniforme, lo que confirma el comportamiento esperado en grafos simétricos.
- **Casos 3 y 4 (Complejo y Hub):** Se establece una jerarquía clara donde los nodos que reciben enlaces de otros nodos importantes obtienen un PageRank significativamente mayor, demostrando la propiedad de “calidad sobre cantidad” del algoritmo.

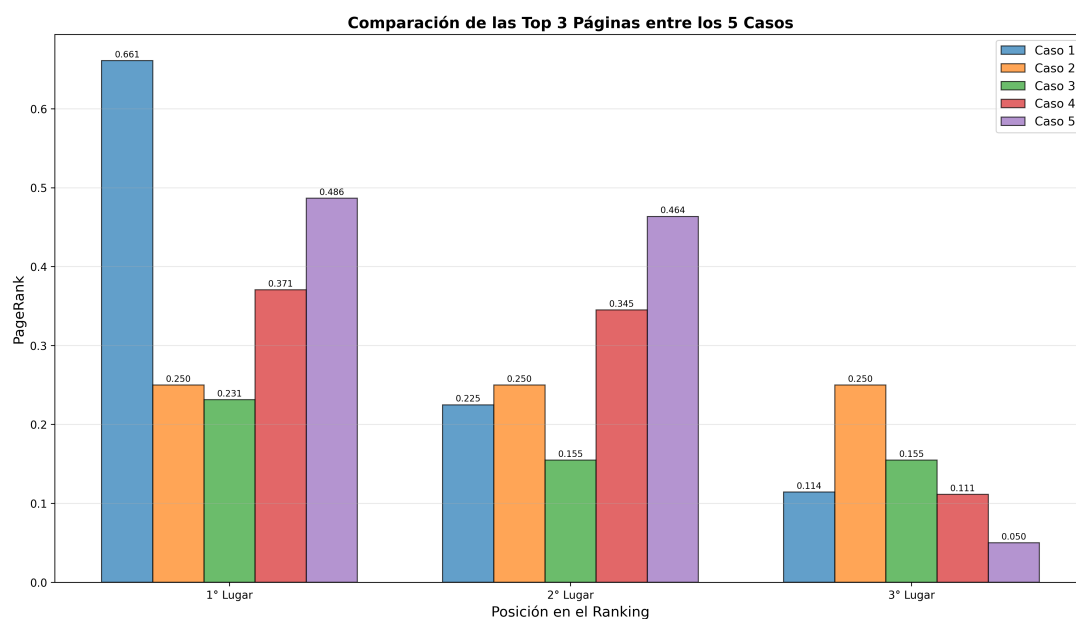


Figura 3: Comparación agrupada del PageRank de los 3 nodos más importantes (Top 3) a través de los 5 casos de prueba.

- En el **Caso 2**, la diferencia entre el 1º y el 3º lugar es nula, reflejando una red descentralizada.
- En los **Casos 1, 4 y 5**, se observa una pendiente negativa pronunciada entre el 1º y el 3º lugar. Esto cuantifica visualmente cómo un solo nodo (el sumidero o el hub) domina la red, mientras que la importancia de los nodos restantes decae rápidamente.
- Esta disparidad valida que el Método de las Potencias, junto con la Matriz de Google, responde sensitivamente a la estructura de enlaces y no asigna valores aleatorios.

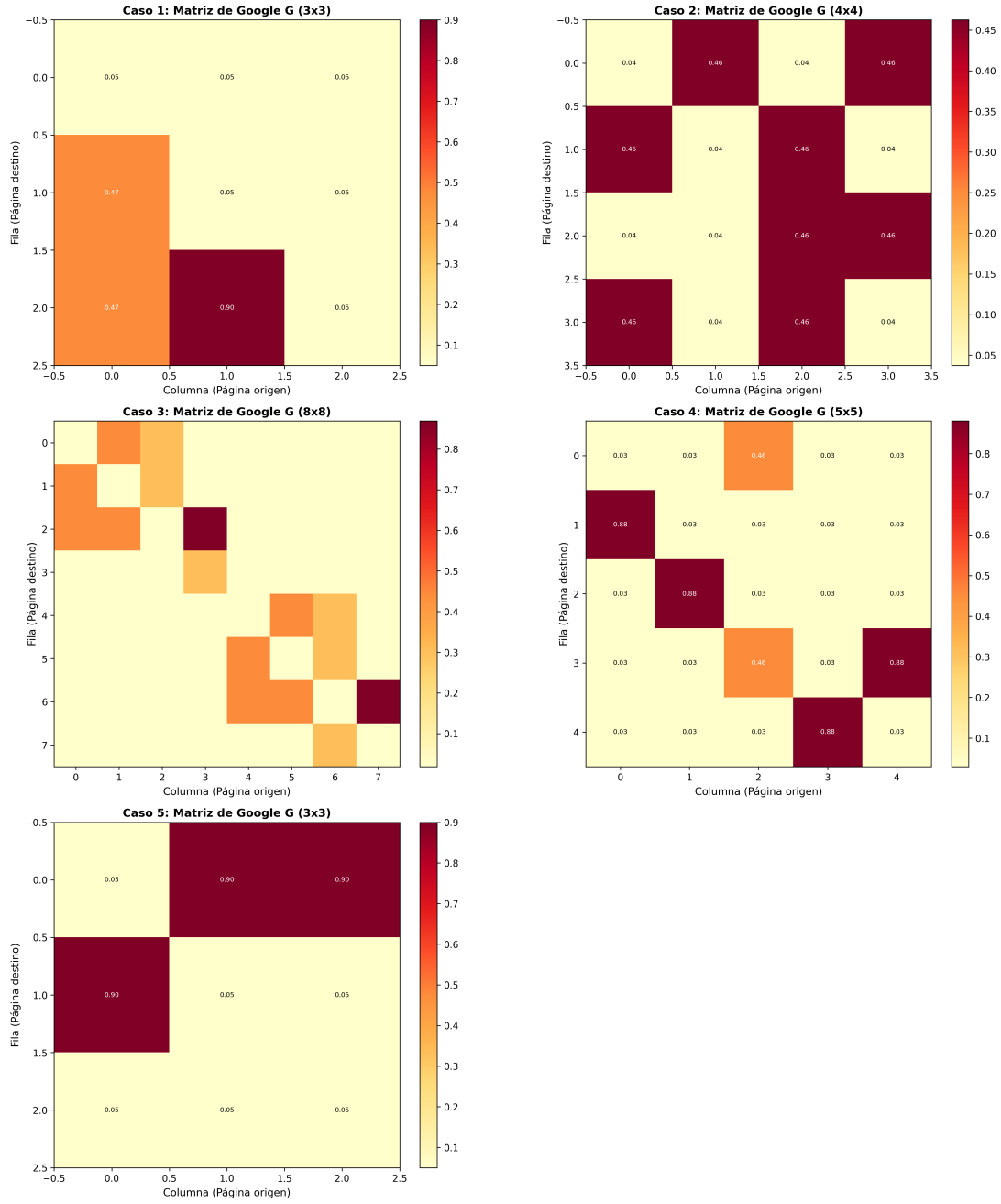


Figura 4: Heatmap de la Matriz de Google G para el Caso 3 (8x8). Se aprecia cómo el término $\frac{1-d}{n}$ llena la matriz con valores base (0.01875), eliminando los ceros de los sumideros.

5. Análisis Matemático y Físico

5.1. Validación de la Suma Unitaria

Una propiedad fundamental del PageRank es que representa una distribución de probabilidad. En todos los casos de prueba, se verificó numéricamente que:

$$\sum_{i=1}^n v_i = 1,0 \pm 10^{-15} \quad (5)$$

Esto confirma que la implementación de la normalización final y la construcción estocástica de G son correctas.

5.2. Efecto del Factor de Amortiguamiento ($d = 0,85$)

Sin el término $\frac{1-d}{n}J$, los Casos 1 y 5 habrían fallado. El nodo sumidero habría atrapado todo el prestigio, y las demás páginas habrían tendido a 0. El factor $d = 0,85$ garantiza que, en cada paso, el 15 % del prestigio se redistribuya uniformemente, “rescatando” al sistema de los sumideros y garantizando la irreducibilidad de G .

5.3. Velocidad de Convergencia

La velocidad de convergencia del Método de las Potencias está gobernada por la razón $\left| \frac{\lambda_2}{\lambda_1} \right| = |\lambda_2|$.

- En el **Caso 2**, $\lambda_2 = 0$, por lo que converge instantáneamente.
- En el **Caso 3**, la topología del grafo hace que $|\lambda_2| \approx 0,98$, requiriendo 451 iteraciones para alcanzar $\epsilon = 10^{-8}$. Esto demuestra que la topología de la red impacta directamente el costo computacional.

6. Conclusiones

1. Se implementó exitosamente el algoritmo PageRank en C++17, logrando una arquitectura modular que separa la lógica matemática (Eigen) de la orquestación y exportación de datos.
2. El uso del factor de amortiguamiento $d = 0,85$ demostró ser crucial. Permitió que los casos con nodos sumidero (Casos 1 y 5) convergieran correctamente, redistribuyendo el prestigio atrapado y garantizando que G fuera una matriz estocástica positiva.
3. El Método de las Potencias probó ser robusto y estable, alcanzando la tolerancia de $\epsilon = 10^{-8}$ en todos los casos. Se validó que el vector resultante siempre suma 1.0, cumpliendo con la definición de distribución de probabilidad estacionaria de una Cadena de Markov.
4. El análisis de los 5 casos confirmó la intuición física del algoritmo: los nodos que reciben enlaces de otros nodos importantes, o que actúan como sumideros, acumulan mayor PageRank, mientras que los grafos regulares distribuyen el prestigio equitativamente.
5. La visualización mediante Python permitió corroborar visualmente el decaimiento exponencial del error y la estructura de la Matriz de Google, cerrando el ciclo de la metodología SDD (Specification-Driven Development).

Referencias

- [1] Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab.

- [2] Langville, A. N., & Meyer, C. D. (2006). *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press.
- [3] Meyer, C. D. (2000). *Matrix Analysis and Applied Linear Algebra*. SIAM. (Capítulo sobre Método de las Potencias y Cadenas de Markov).
- [4] Eigen Documentation. *Dense matrix and array manipulation*. <https://eigen.tuxfamily.org/dox/>